

SHIV NADAR UNIVERSITY CHENNAI

School of Engineering

CS3807 – Deep Learning Laboratory

Experiment – 5

Comparative Study of Deep CNN Training, Regularization, Optimization, Hyperparameter Tuning , Transfer Learning and Cross validation

Name	: Srivatsan S
Register No.	: 24011101093
Degree & Branch	: B.Tech Artificial Intelligence & Data Science
Class / Section	: AIDS – B
Semester	: V
Subject Code	: CS3807
Subject Name	: Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.

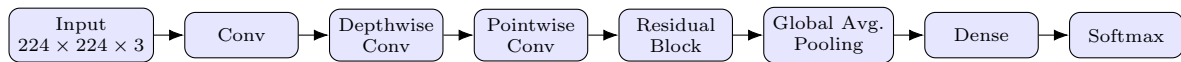
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

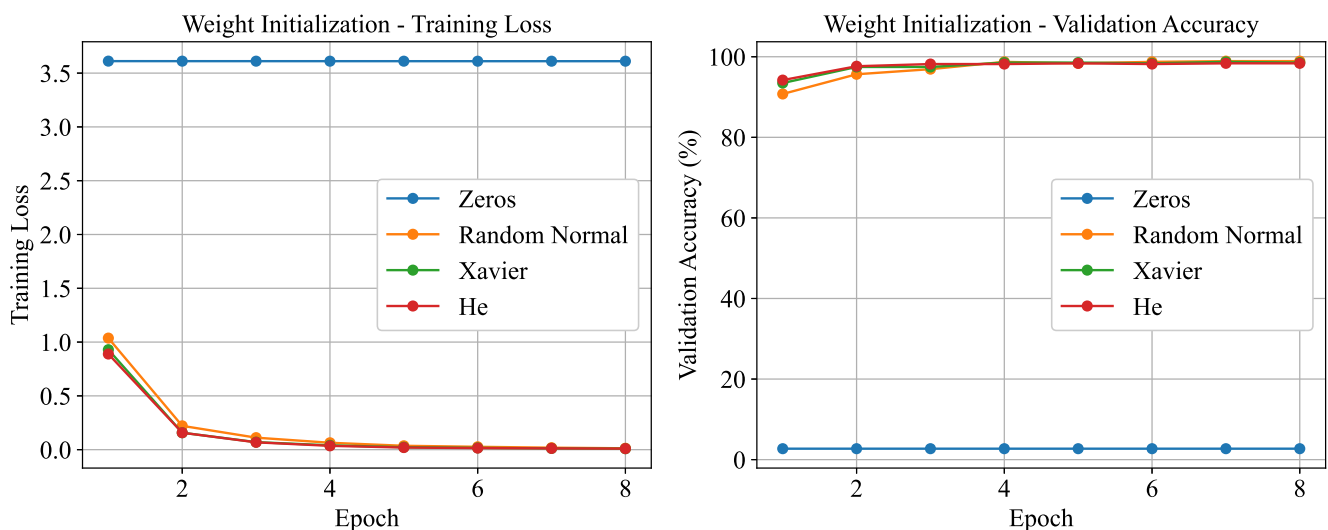
Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Expected Analysis

Students should compare the initialization strategies using:

Plot 1: Training Loss vs. Epoch Plot 2: Validation Accuracy vs. Epoch



Initialising the weights with 0 stays flat both on validation accuracy and training loss at near 0 and near 3.5 respectively. Whereas random Normal, Xavier and He initialisation converges to near 0 loss and achieves accuracy closer to 100. Xavier and He performed slightly better than random normal initialisation.

6. Regularization and Overfitting

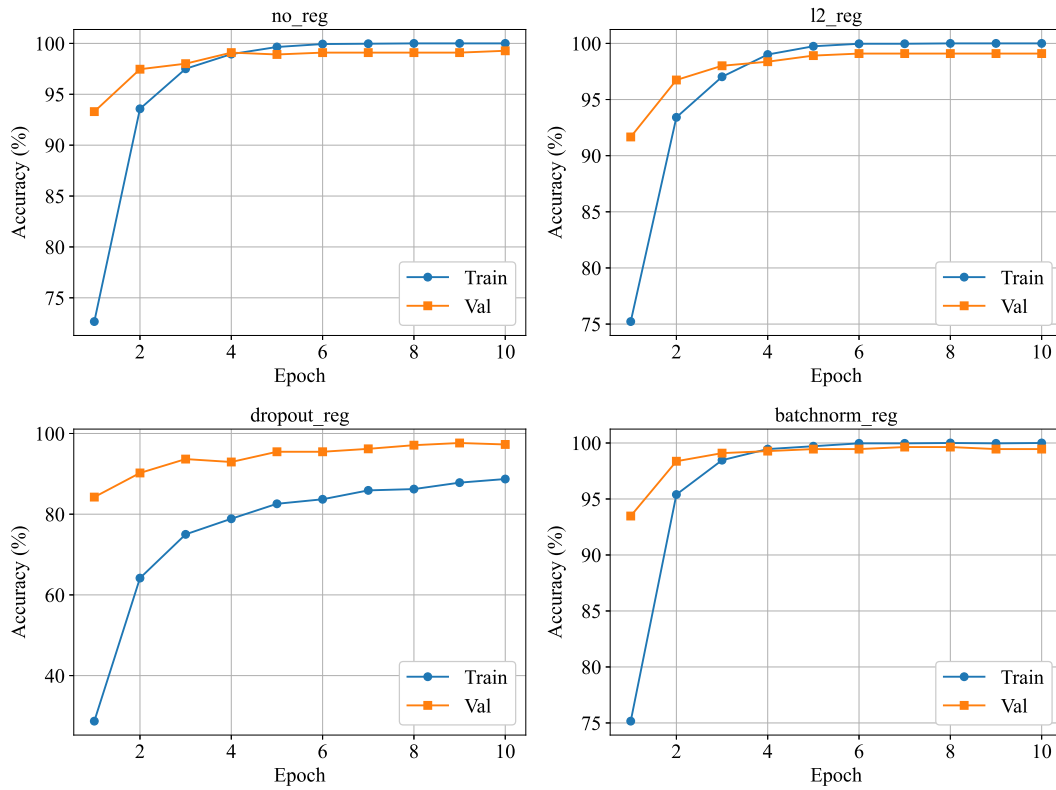
Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data.

Students should compare:

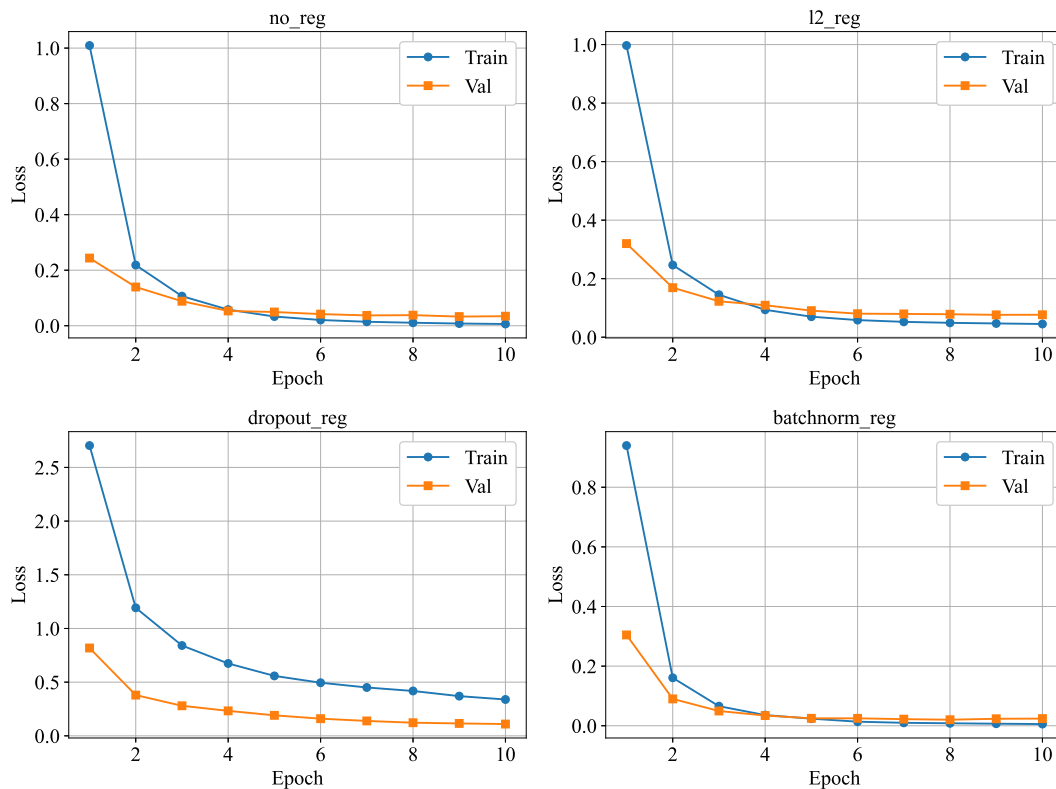
- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Expected Plots

Plot 3: Training and Validation Accuracy vs. Epoch



Plot 4: Training and Validation Loss vs. Epoch



Baseline and l_2 regularization model show a slight gap between training and validation accuracy whereas Batch normalization shows

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training.

For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2 - 5)^2 + (4 - 5)^2 + (6 - 5)^2 + (8 - 5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2 - 5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4 - 5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6 - 5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8 - 5}{2.236} \approx 1.342.$$

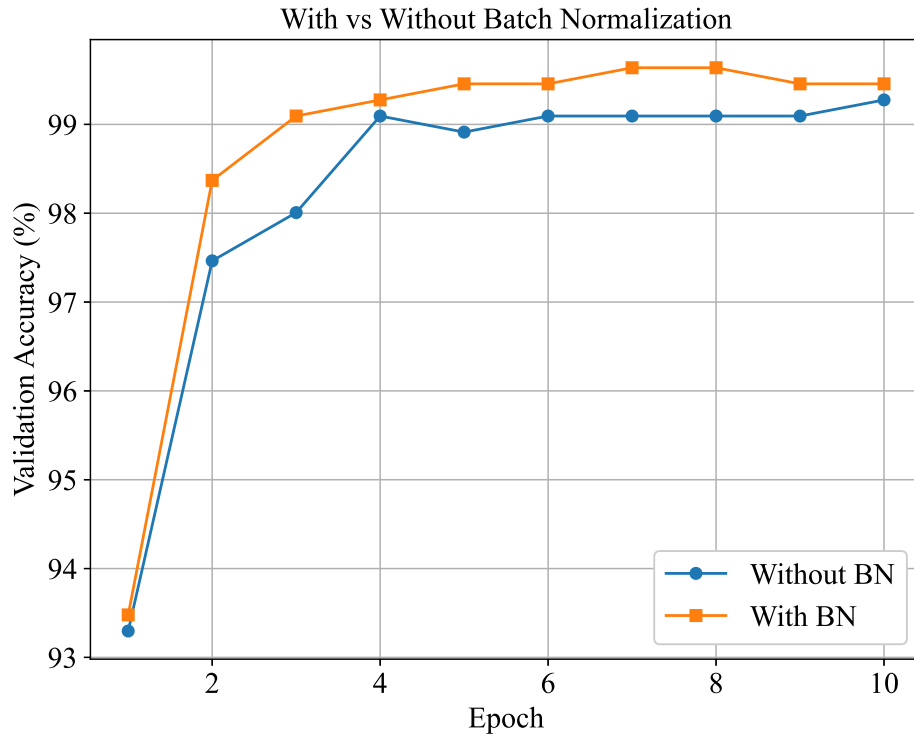
Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

Plot 5: With vs. Without Batch Normalization



Both the curves start similar but the model with batch normalization performs visibly better than the baseline. This is due to the reduction of the internal covariate shift which is why it is comparatively smoother than the baseline.

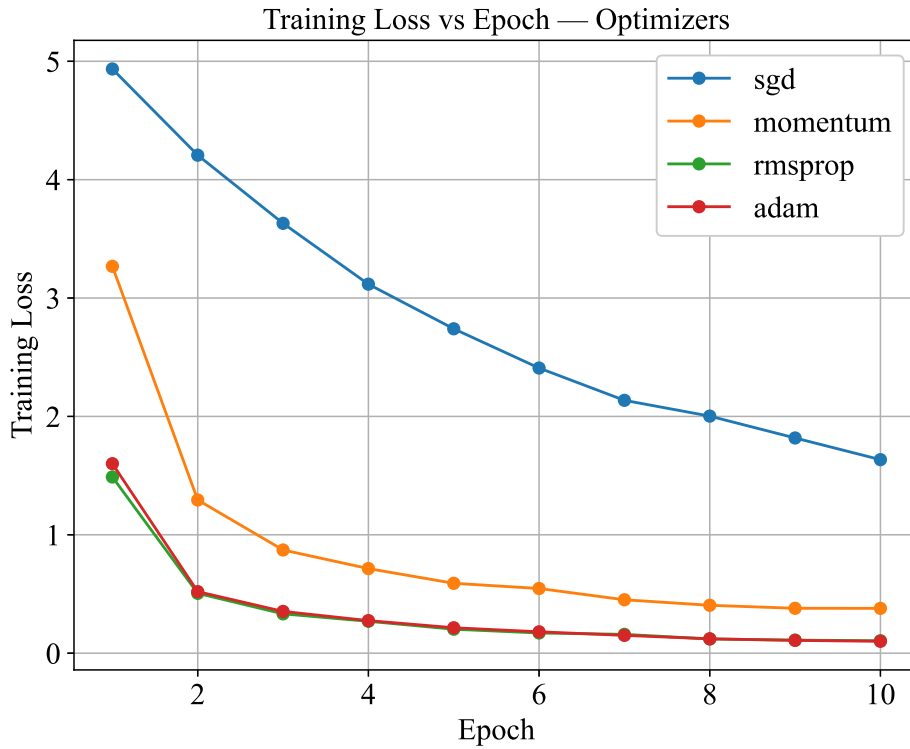
8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

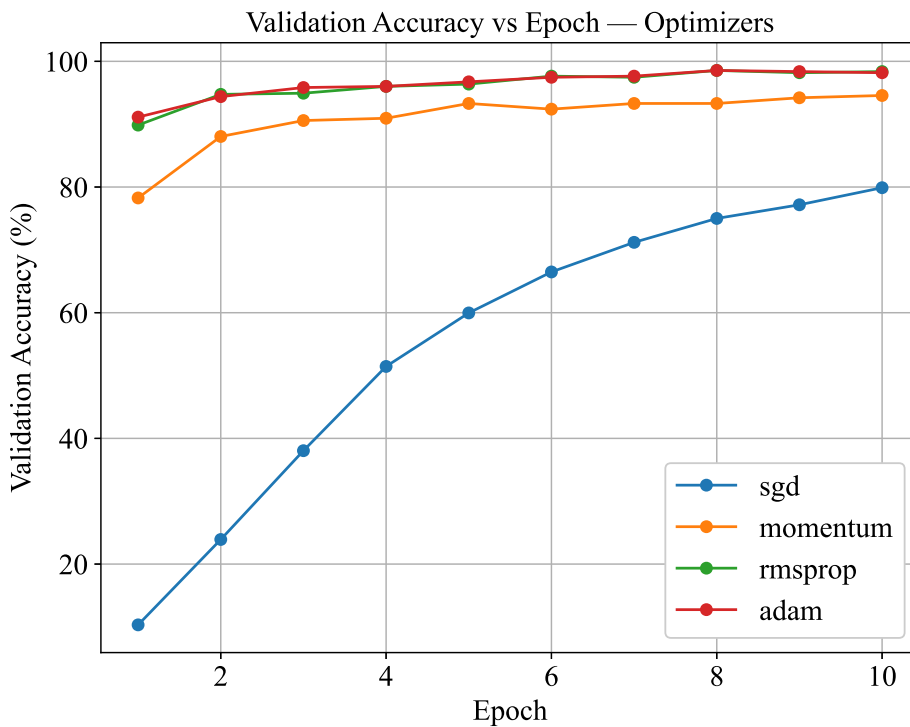
Plot 6: Training Loss vs. Epoch for Different Optimizers



Across all the optimizers sgd descends relatively slowly and has the the worst training loss among the others , Momentum is slightly better than Sgd and RMSProp and ADAM makes the mode's traing loss achieve near 0 value across the 10 epochs

Plot 7: Validation Accuracy vs. Epoch for Different Optimizers

Across all the optimizers sgd ascends relatively slowly and has the the worst validation accuracy among the others , Momentum is slightly better than Sgd and RMSProp and ADAM makes the mode's validation accuracy achieve near 100 value across the 10 epochs



Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	1.6347	79.89%	10	104.9s
Momentum	0.3785	94.57%	10	63.7s
RMSProp	0.1051	98.55%	8	64.0s
Adam	0.1002	98.55%	8	64.9s

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

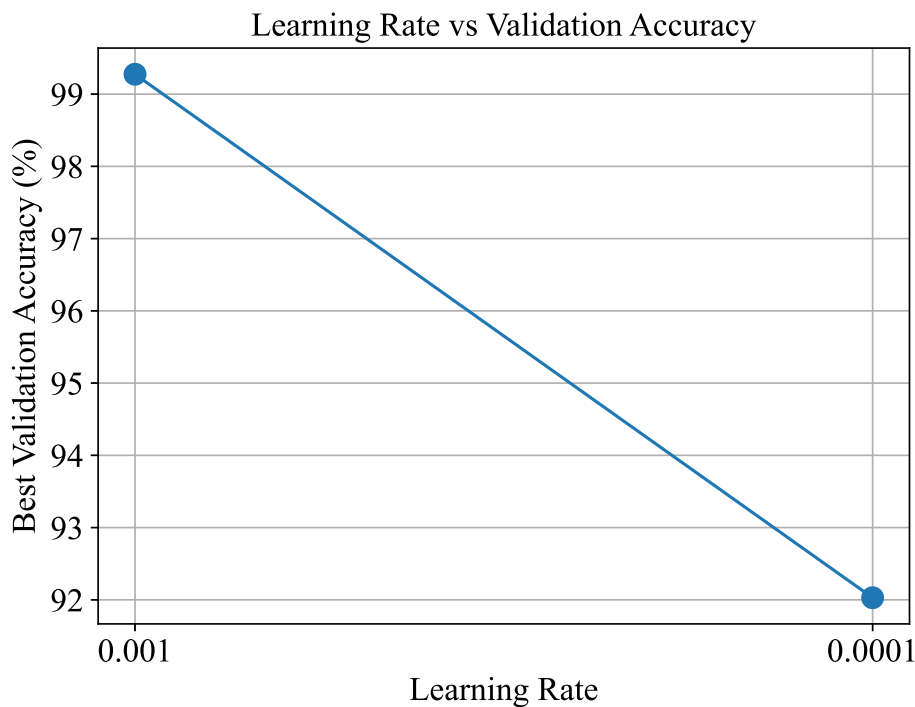
For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

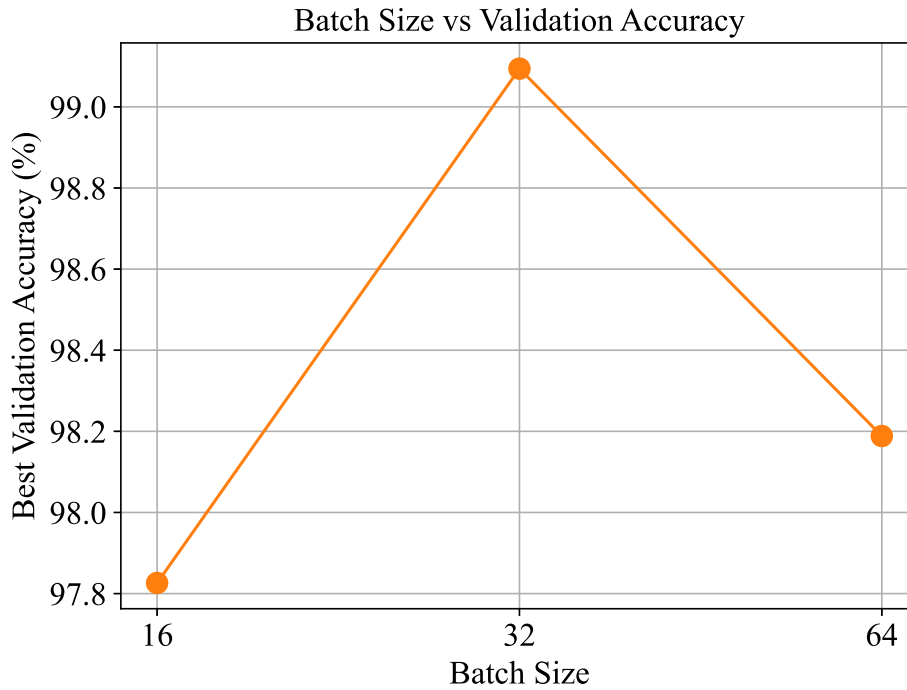
Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

Plot 8: Learning Rate vs. Validation Accuracy



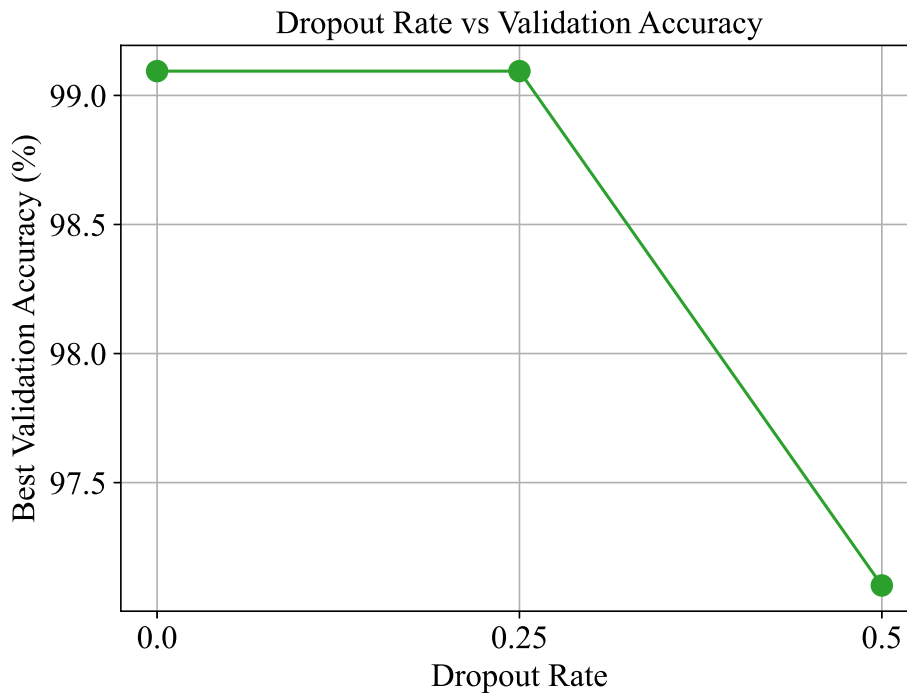
Among the learning rates 0.001 and 0.0001, the former performed better with an accuracy of 99.3 and the latter has an accuracy near 92. This is because the smaller learning rate will require a furthermore epochs to let it allow to converge on the training loss because of the small step size it actually offers

Plot 9: Batch Size vs. Validation Accuracy



The plot shows that the highest accuracy is achieved with the batch size of 32 with accuracy of 99.1 as it provides a balance between a very small batch of 16 (97.8) which gives noisier gradients and a very large batch of 64 (98.2) which reduce the number of gradient updates per epoch

Plot 10: Dropout Rate vs. Validation Accuracy



Both the baseline (0.0) and the dropout of 0.25 has almost the same validation accuracy while dropping out half of the units(0.5) slows convergence under the same number of epochs .

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

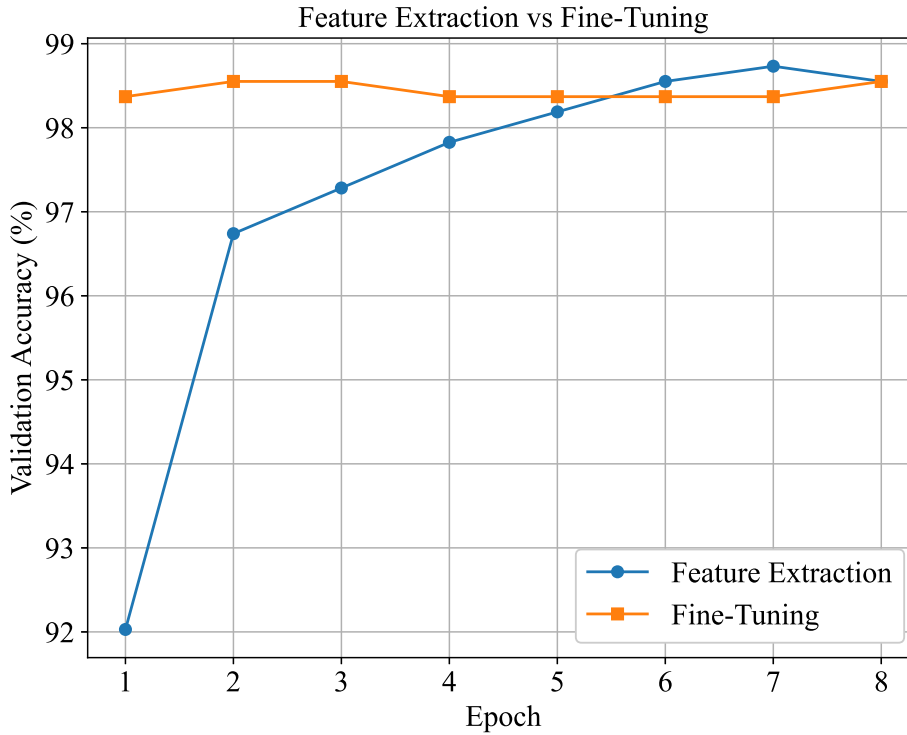
The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

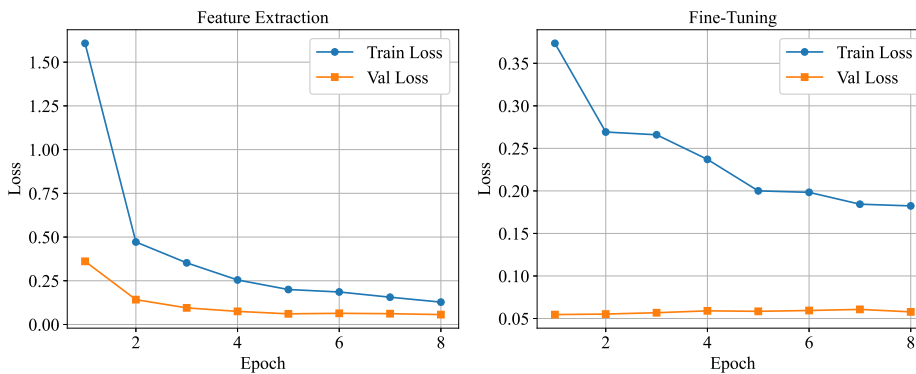
The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

Plot 11: Feature Extraction vs. Fine-Tuning



Extracting features starts lower in terms of validation accuracy (92) and climbs gradually to (98.4) whereas fine tuning converges immediately as it starts from a pretrained model and not from scratch

Plot 12: Training and Validation Loss



Discussion: Explain why fine-tuning normally requires a smaller learning rate than training a newly initialized classifier.

Fine tuning already has a well fit pretrained model and a higher learning rate may cause catastrophic forgetting but to train a new model from scratch we need comparatively large learning rates for faster convergence

11. K-Fold Cross-Validation

After the individual studies, select 3–4 promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

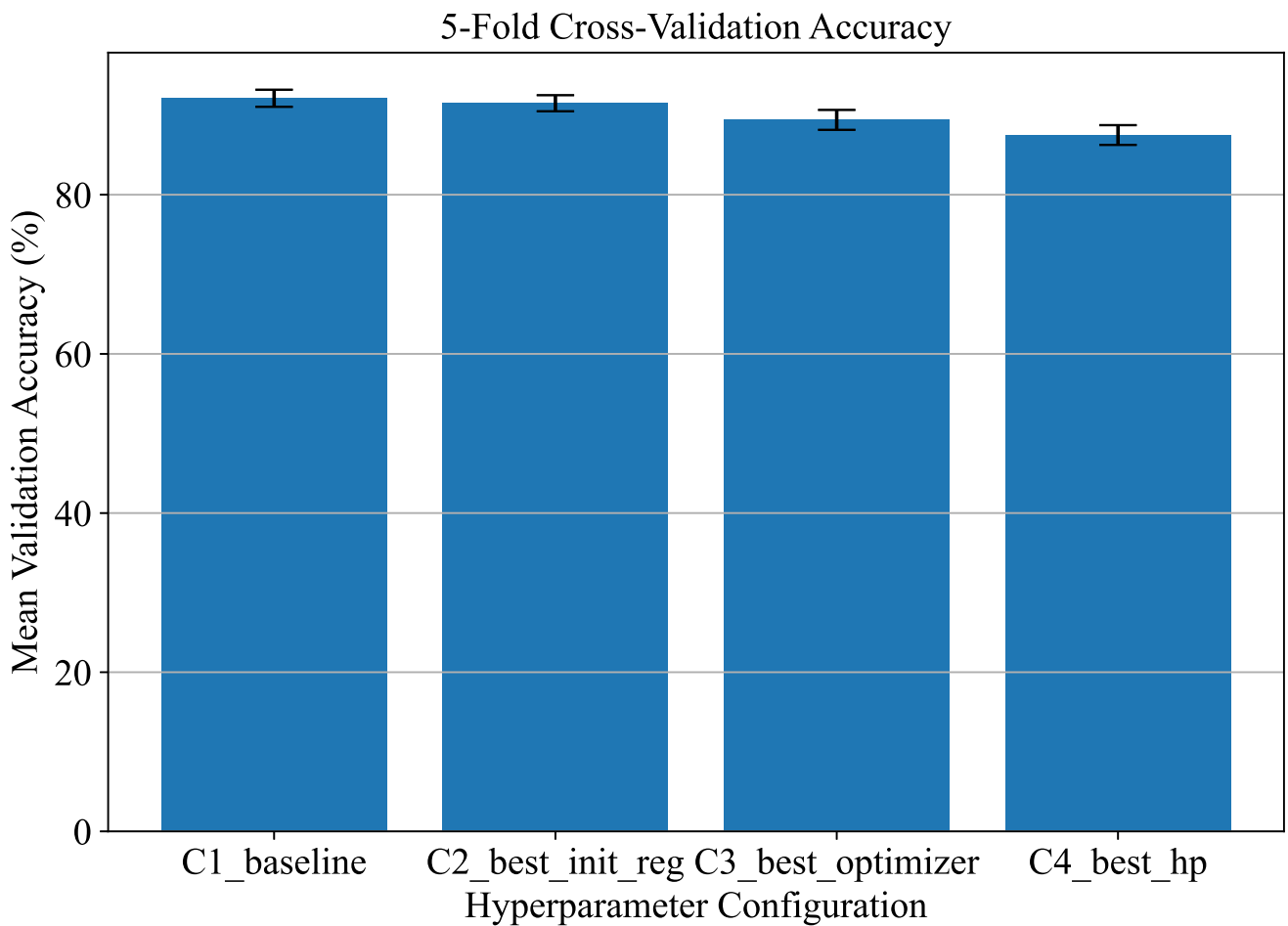
Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
C1	92.26	91.30	93.21	93.07	90.76	92.12 $\pm$ 0.96
C2	91.98	90.62	92.53	92.12	90.22	91.49 $\pm$ 0.90
C3	90.49	88.72	90.90	88.99	87.91	89.40 $\pm$ 1.12
C4	88.04	86.82	89.40	86.28	86.96	87.50 $\pm$ 1.11

Plot 13: 5-Fold Cross-Validation Accuracy



The baseline scores the highest accuracy followed by the best initialiser followed by the best optimizer followed by the model with the best hyperparameters. This is because the subsequent models need few or more epochs to actually show the competence in the increase of the accuracy.

12. Final Model Evaluation

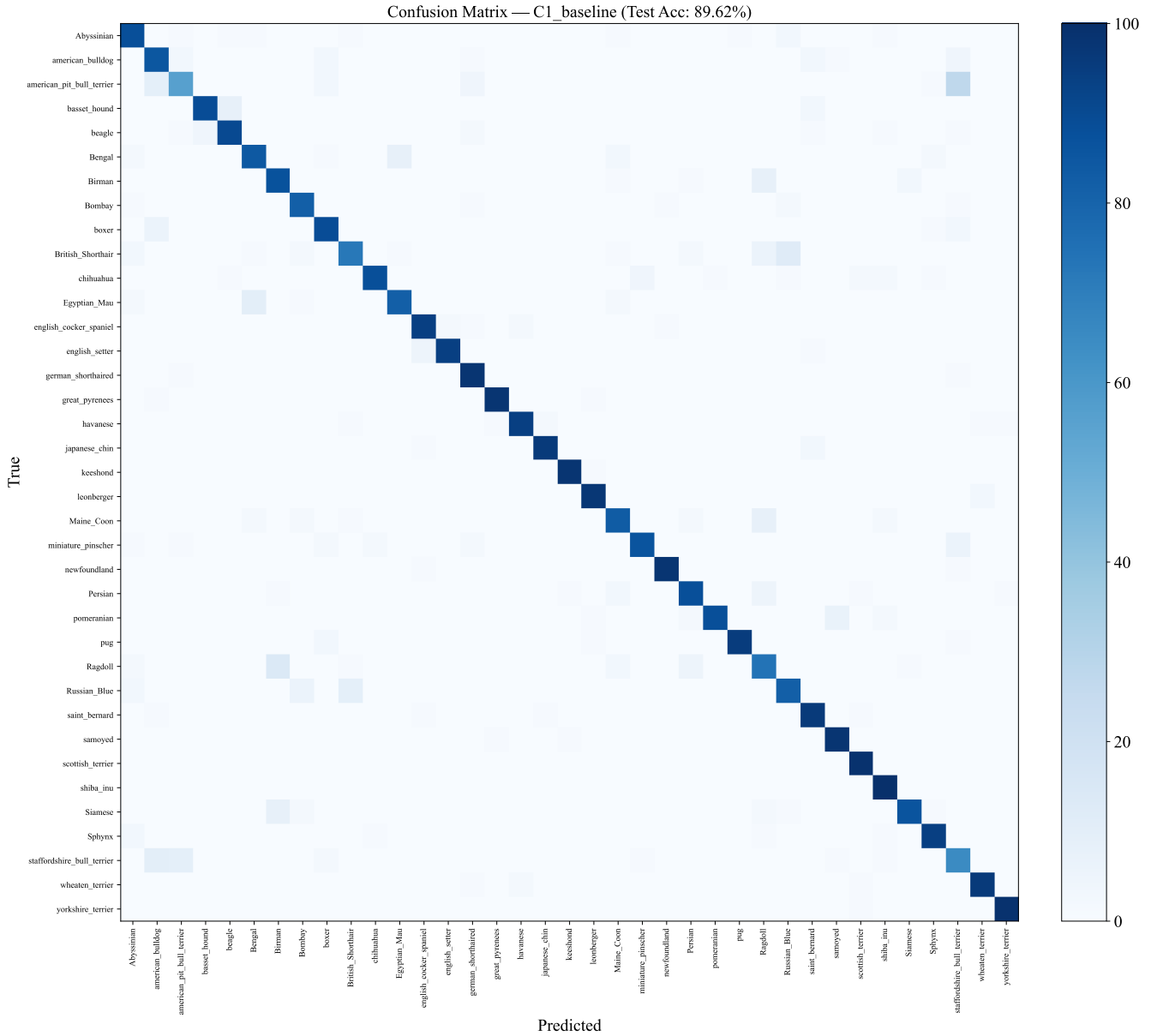
After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	92.12%
CV Standard Deviation	0.96%
Test Accuracy	89.62%
Precision	0.8978
Recall	0.8958
F1-score	0.8952
Training Time	73.6s
Number of Parameters	2,431,845

Plot 14: Confusion Matrix



- the best classified classes; the diagonal elements are predicted the most which signifies the higher accuracy of the model
- the most frequently confused classes; The confusion occurs for similar breed like american and staffordshire bull dogs and cats like the russian blue and british shorthair .
- possible visual reasons for misclassification. Misclassifications occurred for classes with visually similar classes

13. Overall Results

Configuration	CV Accuracy	Test Accuracy	Training Time
Baseline	92.12%	89.62%	73.6s
Best Initialization [†]	98.91%	89.53%	53.4s
Best Regularization [†]	99.64%	90.13%	64.7s
Best Optimizer [†]	98.55%	89.15%	64.0s
Best Hyperparameters [†]	99.09%	86.21%	59.1s
Fine-Tuned Model [†]	98.55%	89.81%	73.8s

[†]Single train/val split result (not 5-fold CV).

14. Discussion Questions

- **What is the difference between model parameters and hyperparameters?**

Model parameters are values that the model learns automatically from the training data, such as weights and biases. Hyperparameters are values that are set before or during training that control the learning process, such as learning rate, batch size, number of epochs, dropout rate, and weight initialization method.

- **Why is weight initialization important?**

Weight initialization gives the starting values for the model parameters. A good initialization helps gradients flow properly and allows the network to converge faster and more reliably. A bad initialization can lead to vanishing or exploding gradients or slow learning.

- **Why can zero initialization be problematic for networks?**

If all weights are set to zero, neurons in the layer get the same gradients and stay the same during training. This breaks symmetry, stopping neurons from learning different features. As a result, the network has limited learning power.

- **Compare Xavier and He initialization.**

Xavier initialization is meant to keep the variance of activations and gradients stable and is often used with activation functions like sigmoid or tanh. He initialization uses a variance suited for ReLU-based networks. Since the model uses ReLU activations, He initialization is usually better.

- **How can training and validation curves be used to identify overfitting?**

Overfitting happens when training performance keeps getting better while validation performance stops improving or gets worse. For example, if training loss keeps dropping while validation loss starts rising, the model is likely memorizing the training data instead of learning to generalize well.

- **How does Dropout reduce overfitting?**

Dropout randomly turns off a part of the neurons during training. This stops the network from depending too much on specific neurons and helps it learn more robust and distributed features, which reduces overfitting.

- **What is the purpose of Batch Normalization?**

Batch Normalization makes the activations of a layer stable by using the mean and variance of a mini-batch. This helps make training more stable, allows suitable learning rates to be used, and can improve how fast the model converges and how well it generalizes.

- **What are the roles of γ and β ?**

γ is a scaling parameter and β is a learnable shifting parameter. They let the network learn the scale and offset after normalization. Therefore, Batch Normalization does not force the layer to use zero mean and unit variance.

- **Compare SGD, Momentum, RMSProp and Adam.**

SGD updates parameters directly using the gradient and learning rate. Momentum adds past gradients to help move in consistent directions and reduce shaking. RMSProp changes the learning rate for each parameter using a moving average of gradients. Adam uses momentum-like first-moment estimation and RMSProp-like second-moment estimation, giving adaptive learning rates and usually fast convergence.

- **What happens when the learning rate is too large?**

The parameter updates are too big. The model might jump over the minimum, shake around it, fail to converge, or even make the loss get worse.

- **What happens when the learning rate is too small?**

The parameter updates are very small. Training takes a long time and the model might need many more epochs to converge. It can also be hard to get out of regions in the optimization landscape.

- **What is the effect of increasing batch size?**

A larger batch gives a more stable estimate of the gradient and can make training faster on suitable hardware. However, it needs more memory and can reduce the noise in gradient updates, which can sometimes affect how well the model generalizes.

- **Explain stride and padding.**

Stride is how many pixels the convolution filter moves each time. A stride of 1 moves one pixel at a time, while a larger stride reduces the size more quickly. Padding adds pixels, usually zeros, around the input. It helps keep the size and lets edge information be processed.

- **Why is MobileNetV2 computationally efficient?**

MobileNetV2 is built for computational efficiency and uses inverted residual blocks with linear bottlenecks and depthwise separable convolutions. These greatly cut down on the number of calculations and parameters compared to standard convolutions while keeping useful feature representations.

- **What is depthwise separable convolution?**

Depthwise separable convolution splits a convolution into two steps. A depthwise convolution applies a filter to each input channel separately, then a pointwise 1×1 convolution mixes information across channels. This takes fewer calculations than a normal convolution.

- **What is transfer learning?**

Transfer learning uses knowledge a model has learned on a source dataset and applies it to a different target task. In this experiment, MobileNetV2 trained on ImageNet is used as the base network for the Oxford-IIIT Pet classification task.

- **Differentiate feature extraction and fine-tuning.**

In feature extraction, the pretrained base model is kept frozen. Only the new classification layers are trained. In fine-tuning, some or all of the pretrained layers are unfrozen and trained along with the new layers so the learned features can fit the new data.

- **Why is a smaller learning rate generally used during fine-tuning?**

The pretrained model already has useful features. A large learning rate might change these weights too much and destroy the useful features, which is called catastrophic forgetting. A smaller learning rate lets the features adapt slowly to the task.

- **Why is K-Fold Cross-Validation useful for hyperparameter selection?**

K-Fold Cross-Validation splits the training data into K parts and trains and tests the model multiple times using different parts as validation data. It gives an idea of how a configuration performs across different parts of the data and helps choose hyperparameters that work well overall, rather than just on one split.

- **Why must the test set remain untouched during tuning?**

The test set is supposed to give an idea of the model's final performance. If it is used repeatedly during hyperparameter selection, information from the test set might influence the model choice. The test accuracy might then be too high and not be a true measure of generalization.

- **Why should mean and standard deviation both be reported?**

The mean shows the performance across folds, while the standard deviation shows how much the performance changes between folds. Reporting both gives information about both performance and consistency. A model with a lower mean but much lower standard deviation might be more reliable.

- **Is the highest validation accuracy always sufficient to select a model?**

No. Model selection needs more than the highest validation accuracy. Mean performance, standard deviation, overfitting, cost, complexity, and generalization should also be considered. A model with lower accuracy but more stability or much lower cost might be better.

15. SOURCE CODE

<https://github.com/sriwhatsnext/dllab/tree/main/Week05>

16. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>